

Analyse de la CVE-2024-23724 : Vulnérabilité dans Ghost CMS

Youssef Ichchou, Yassine Safraoui, Ilyas Lok-Ayba

Janvier 18, 2025

Table des matières

0.1	Introduction	1
0.2	Analyse de la vulnérabilité	2
0.2.1	Description technique	2
0.2.2	Architecture typique affectée	2
0.3	Mesures de sécurisation contre la vulnérabilité CVE-2024-23724 dans Ghost CMS	3
0.3.1	Sanitisation des fichiers SVG	4
0.3.2	Contrôle des types de fichiers autorisés	4
0.3.3	Privilégier l'utilisation de mécanismes d'authentification et d'autorisation robustes	4
0.3.4	Surveillance et analyse des fichiers téléchargés	5
0.3.5	Mesures spécifiques pour prévenir l'exploitation via l'ouverture dans un nouvel onglet	5
0.3.6	Conclusion	5
0.4	Installation	5
0.4.1	Installation automatique (recommandée)	5
0.4.2	Installation manuelle avec Mailgun	6
0.4.3	Tester l'attaque	7
0.5	Conclusion	8

Résumé

Ce rapport examine en détail la vulnérabilité CVE-2024-23724, identifiée dans Ghost CMS jusqu'à la version 5.76.0. Cette faille permet l'exécution de scripts intersites (XSS) stockés, entraînant une élévation de privilèges où un contributeur peut prendre le contrôle de n'importe quel compte via une image de profil SVG contenant du code JavaScript malveillant. Ce document explore les mécanismes de cette vulnérabilité, les architectures affectées, et propose des stratégies pour la détecter et s'en protéger.

0.1 Introduction

La gestion des vulnérabilités constitue un aspect crucial de la sécurité informatique moderne. Dans un environnement numérique où les attaques sont en constante évolution, il est impératif de comprendre et d'atténuer les risques associés aux vulnérabilités. Le présent rapport s'intéresse à la CVE-2024-23724, une vulnérabilité critique affectant Ghost CMS, une plateforme populaire pour la gestion de contenu.

Cette vulnérabilité permet l'exécution de scripts malveillants via l'utilisation d'images SVG contenant du code JavaScript. Exploitée de manière malveillante, elle peut conduire à une escalade de privilèges où un utilisateur ayant un rôle limité, comme celui de contributeur, peut obtenir les droits d'administrateur ou même propriétaire. Cette faille souligne l'importance de la validation des entrées utilisateur et de la gestion rigoureuse des fichiers envoyés par des tiers.

Le rapport se propose d'analyser en détail cette vulnérabilité, d'explorer les architectures potentiellement affectées, et de présenter des mesures de mitigation pour réduire les risques. En outre, une méthode de test pratique est également documentée afin de permettre aux administrateurs et chercheurs en sécurité de reproduire et de comprendre pleinement cette menace.

En examinant cette vulnérabilité, ce rapport vise à fournir des éléments précis et exploitables pour renforcer la sécurité des systèmes déployant Ghost CMS. La démarche adoptée met l'accent sur une compréhension approfondie du fonctionnement interne de la faille, ainsi que sur des recommandations pratiques pour limiter son impact.

0.2 Analyse de la vulnérabilité

0.2.1 Description technique

La vulnérabilité **CVE-2024-23724** affecte le système de gestion de contenu **Ghost CMS**, jusqu'à la version 5.76.0, et permet l'exécution de scripts intersites (XSS) stockés. Cette vulnérabilité est causée par la manière dont **Ghost CMS** traite les fichiers **SVG** (Scalable Vector Graphics) lorsqu'ils sont utilisés comme images de profil des utilisateurs.

Les fichiers **SVG** sont un format d'image vectorielle populaire en raison de leur flexibilité et de leur capacité à être redimensionnés sans perte de qualité. Cependant, contrairement à d'autres formats d'images comme **JPEG** ou **PNG**, les fichiers **SVG** peuvent contenir du code JavaScript intégré, qui est interprété et exécuté lorsque l'image est intégrée directement dans le code de la page web. Cela ouvre la porte à une vulnérabilité de type **XSS** (Cross-Site Scripting), dans laquelle du code malveillant peut être injecté dans un fichier **SVG** et exécuté automatiquement dans l'environnement de l'utilisateur.

Dans le cas de **Ghost CMS**, l'attaque commence lorsqu'un utilisateur ouvre un fichier **SVG** malveillant dans un nouvel onglet de son navigateur. Une fois ouvert, ce fichier malveillant interagit avec l'API locale de **Ghost CMS**, qui est généralement accessible via `http://localhost:3001/ghost/api`. Cette API est utilisée pour gérer divers aspects du site, notamment les utilisateurs, les articles, les paramètres du système, et d'autres informations sensibles.

En exploitant cette vulnérabilité, un attaquant peut envoyer des requêtes non autorisées à l'API locale de **Ghost CMS**. Ces requêtes malveillantes peuvent permettre à l'attaquant de réaliser plusieurs actions dangereuses, telles que :

- La modification des privilèges des utilisateurs : Un attaquant pourrait, par exemple, s'attribuer des privilèges d'administrateur ou de propriétaire, ou manipuler les droits d'accès à certaines parties du site.
- La prise de contrôle des comptes administrateurs : En obtenant des privilèges d'administrateur, l'attaquant peut modifier la configuration du système, accéder à des données sensibles, ou même supprimer des contenus.
- L'envoi de requêtes malveillantes pour voler des données sensibles des utilisateurs ou pour altérer l'intégrité du site.

L'impact de cette vulnérabilité peut être grave, notamment dans des environnements où **Ghost CMS** est utilisé pour gérer des sites web avec des données sensibles ou des utilisateurs ayant des privilèges élevés.

La possibilité d'exécuter des scripts malveillants via des fichiers **SVG** souligne la nécessité de renforcer les contrôles de sécurité autour du téléchargement et de l'affichage des fichiers sur les plateformes web. Il est essentiel que les systèmes comme **Ghost CMS** vérifient rigoureusement les fichiers téléchargés, en particulier les fichiers **SVG**, et qu'ils les nettoient de toute trace de code malveillant avant de les afficher aux utilisateurs.

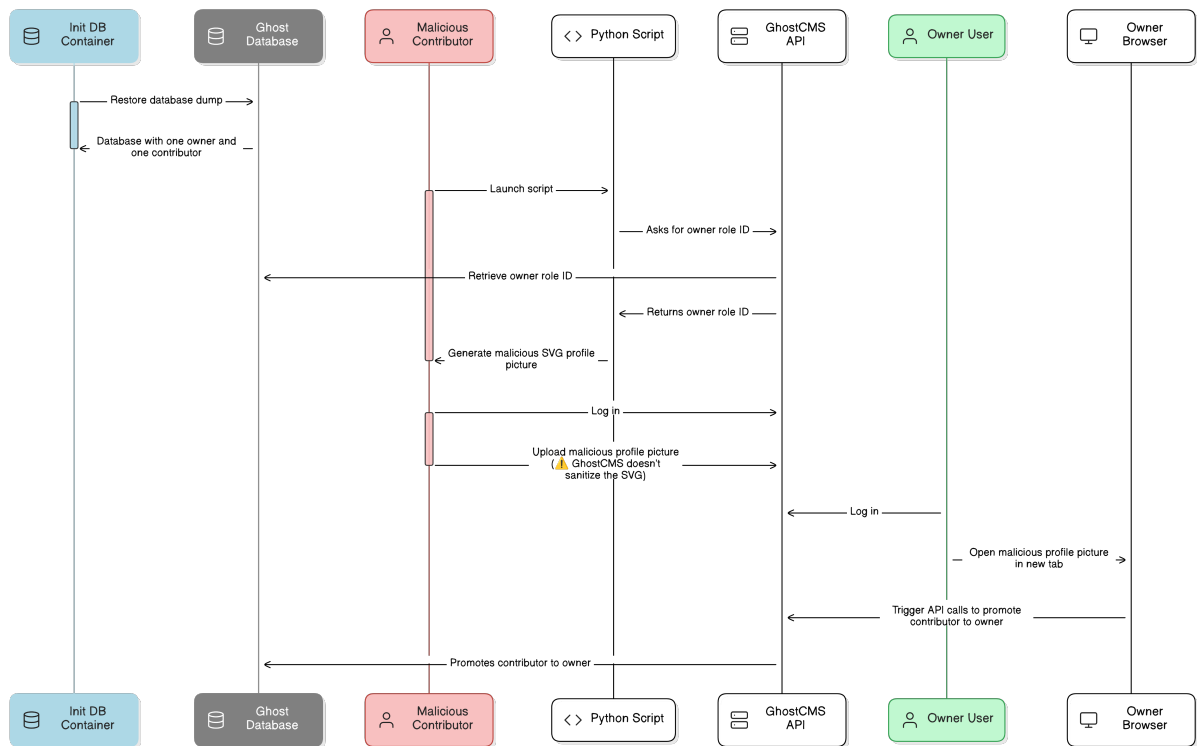
0.2.2 Architecture typique affectée

Ghost CMS est couramment déployé dans des environnements cloud ou sur des serveurs dédiés. L'architecture typique de **Ghost CMS** comprend plusieurs composants interconnectés :

- Un serveur d'application exécutant **Ghost CMS**, qui gère l'affichage du contenu, la gestion des utilisateurs, et l'interaction avec les autres services
- Une base de données (généralement MySQL ou SQLite) pour stocker les informations utilisateur, les articles, les paramètres du site, et autres contenus.
- Un serveur de messagerie pour gérer l'envoi de notifications, d'invitations par e-mail et d'autres communications avec les utilisateurs.
- Des utilisateurs finaux qui accèdent au système via des navigateurs web pour interagir avec le contenu, publier des articles, ou gérer leur profil.

Dans cette architecture, la vulnérabilité affecte principalement l'interaction entre le serveur d'application et la base de données, ainsi que les mécanismes de gestion des utilisateurs. L'API locale de **Ghost CMS**, si elle est compromise, peut permettre à un attaquant de manipuler ces composants et d'exécuter des actions non autorisées.

Attack Sequence on GhostCMS



0.3 Mesures de sécurisation contre la vulnérabilité CVE-2024-23724 dans Ghost CMS

La vulnérabilité CVE-2024-23724, qui affecte Ghost CMS, est liée à une faille XSS (Cross-Site Scripting) permettant l'exécution de scripts malveillants via le téléchargement de fichiers SVG non sécurisés. Cette vulnérabilité ne devient effective que lorsque l'utilisateur ouvre le fichier SVG dans un nouvel onglet, où le script malveillant est exécuté. Bien que le fournisseur ait estimé que cette vulnérabilité ne constitue pas un vecteur d'attaque valide, des mesures spécifiques doivent être prises pour sécuriser les utilisateurs contre cette menace.

0.3.1 Sanitisation des fichiers SVG

La principale cause de la vulnérabilité réside dans l'absence de validation et de nettoyage des fichiers SVG lors de leur téléchargement. Ainsi, la première mesure de sécurisation consiste à s'assurer que tous les fichiers SVG téléchargés sont soigneusement traités avant d'être enregistrés sur le serveur.

- **Utilisation de DOMPurify** : DOMPurify est une bibliothèque qui permet de nettoyer les fichiers SVG en supprimant tout code JavaScript malveillant. Cette approche garantit que les fichiers SVG restent fonctionnels (conservation des propriétés comme les couleurs et les dimensions) tout en éliminant toute menace potentielle d'exécution de script.
- **Mise en œuvre de la sanitisation côté serveur** : Il est recommandé d'intégrer DOMPurify directement dans le processus de téléchargement des fichiers, afin de valider et nettoyer tous les fichiers SVG avant leur stockage ou leur utilisation dans l'application. Cela peut être réalisé en modifiant le code du middleware de téléchargement, en particulier le fichier `/ghost/core/core/server/web/api/middleware/upload.js`, pour y ajouter une étape de nettoyage des fichiers SVG.

0.3.2 Contrôle des types de fichiers autorisés

Une autre mesure importante consiste à restreindre les types de fichiers autorisés au téléchargement. Bien que Ghost CMS permette par défaut l'upload de fichiers SVG pour les profils d'utilisateurs, il est fortement recommandé de ne permettre que les types de fichiers strictement nécessaires à l'application, et de désactiver l'upload de formats risqués (comme les fichiers contenant des scripts).

- **Restriction des formats de fichiers** : Limiter les types de fichiers autorisés à ceux jugés sûrs, tels que les images PNG, JPG, ou d'autres formats standards.
- **Validation côté serveur des extensions et contenus** : En plus de valider les extensions des fichiers téléchargés, il est essentiel de vérifier leur contenu à l'aide d'outils spécialisés pour s'assurer qu'ils ne contiennent pas de code malveillant.

0.3.3 Privilégier l'utilisation de mécanismes d'authentification et d'autorisation robustes

Comme la vulnérabilité nécessite qu'un utilisateur malveillant ait accès au système pour télécharger des fichiers SVG infectés, il est essentiel de mettre en place des contrôles d'accès rigoureux pour les utilisateurs qui peuvent télécharger du contenu, en particulier dans des zones sensibles de l'application.

- **Mise en place de l'authentification forte** : Utiliser des mécanismes d'authentification à deux facteurs (2FA) pour les utilisateurs ayant des privilèges de création de contenu ou de gestion des fichiers et pas seulement les utilisateurs ayant des privilèges élevés.
- **Séparation des rôles** : S'assurer que seules les personnes de confiance ont accès à la fonctionnalité de téléchargement de fichiers et que les permissions sont strictement contrôlées.

0.3.4 Surveillance et analyse des fichiers téléchargés

Enfin, il est important de mettre en place des mécanismes de surveillance pour détecter toute activité suspecte liée aux fichiers téléchargés, notamment l'ajout de fichiers SVG dans des contextes où ils ne sont pas nécessaires.

- **Surveillance des fichiers téléchargés** : Implémenter des outils de monitoring qui suivent les fichiers téléchargés et signalent toute anomalie.
- **Analyse régulière des fichiers SVG** : Effectuer des analyses régulières sur les fichiers SVG téléchargés afin de détecter toute tentative d'exécution de scripts malveillants.

0.3.5 Mesures spécifiques pour prévenir l'exploitation via l'ouverture dans un nouvel onglet

L'attaque XSS via les fichiers SVG nécessite que l'utilisateur ouvre le fichier dans un nouvel onglet. Pour empêcher ce comportement, plusieurs solutions peuvent être mises en place :

- **Restreindre les possibilités d'ouverture des fichiers dans un nouvel onglet** :
Modifiez les configurations de l'application pour interdire ou limiter les actions permettant aux utilisateurs de visualiser des fichiers SVG dans un nouvel onglet. Par exemple, vous pouvez utiliser des liens contenant des attributs `target="_blank"` pour les empêcher d'ouvrir des fichiers SVG directement dans un nouvel onglet.
- **Affichage des fichiers SVG dans un environnement contrôlé** :
Au lieu de permettre à un utilisateur de télécharger et d'ouvrir directement un fichier SVG, vous pouvez forcer son affichage dans un environnement sécurisé où le contenu est immédiatement nettoyé, évitant ainsi toute tentative d'exécution malveillante.

0.3.6 Conclusion

Bien que Ghost CMS n'ait pas jugé cette vulnérabilité comme étant un vecteur d'attaque valide, il est crucial de mettre en place ces mesures de sécurisation pour protéger les utilisateurs contre les risques d'attaques XSS via des fichiers SVG. L'intégration de DOM-Purify pour nettoyer les fichiers SVG, la mise en place de contrôles d'accès appropriés, de mécanismes de surveillance, ainsi que les restrictions d'ouverture des fichiers dans un nouvel onglet permettent de réduire considérablement le risque d'exploitation de cette vulnérabilité.

0.4 Installation

0.4.1 Installation automatique (recommandée)

Pour tester la vulnérabilité, deux comptes sont nécessaires : un compte avec le rôle d'administrateur (propriétaire) et un compte ayant le rôle de contributeur. Dans Ghost CMS, il est nécessaire d'abord de créer le compte propriétaire via le panneau d'administration, puis d'inviter un contributeur pour qu'il crée son propre compte. Cependant, dans Ghost, la seule méthode pour inviter un contributeur consiste à envoyer un lien d'invita-

tion par e-mail, ce qui nécessite un service de livraison d'e-mails pour envoyer l'invitation au contributeur.

Afin de simplifier cette configuration, nous avons utilisé un compte **Mailgun**, ajouté un contributeur à **Ghost**, et créé un dump de la base de données dans l'état où nous l'avions. Dans notre configuration **Docker Compose**, nous restaurons d'abord la base de données à partir du dump avant de démarrer **Ghost**, ce qui permet de tester la vulnérabilité sans avoir besoin d'un service de livraison d'e-mails pour pouvoir créer l'utilisateur contributeur. Toutefois, si vous préférez effectuer une installation propre sans utiliser le dump de base de données fourni, vous pouvez utiliser la configuration **Docker Compose** dans le dossier `manual-setup` pour démarrer une installation **Ghost** propre.

Voici les identifiants à utiliser pour se connecter aux comptes :

- **Admin :**
Email : `blog.boss@company.com`
Mot de passe : `ZtXfMq2FaRHGm@!`
- **Contributeur :**
Email : `malicious.contributor@company.com`
Mot de passe : `ZtXfMq2FaRHGm@!`

0.4.2 Installation manuelle avec Mailgun

Pour démarrer, vous devez créer un compte sur **Mailgun**. Une fois le compte créé, obtenez vos identifiants SMTP. Ensuite, vous devrez ajouter ces identifiants dans le fichier `config.development.json` en suivant le format du fichier `config.example.json`.

Voici un guide étape par étape :

1. Créez un compte sur **Mailgun**.
2. Accédez à la section des identifiants SMTP dans votre tableau de bord Mailgun.
3. Copiez vos identifiants SMTP.
4. Dupliquez le fichier `config.example.json` et renommez-le en `config.development.json`.
5. Remplissez vos identifiants SMTP dans le fichier `config.development.json`. Cela garantit que votre environnement de développement est correctement configuré pour envoyer des e-mails via **Mailgun**.
6. Si vous avez créé un compte sur **Mailgun**, le domaine Mailgun par défaut est un domaine **sandbox**, ce qui signifie que vous devez autoriser l'email du contributeur que vous souhaitez utiliser pour recevoir le lien d'invitation. Vous pouvez faire cela depuis le tableau de bord Mailgun sous (**barre latérale gauche**) **Send > Sending > Overview**. En effet, **Mailgun** n'envoiera des e-mails qu'aux destinataires autorisés lorsque vous utilisez un domaine **sandbox**. Une fois le destinataire autorisé ajouté, vous recevrez un e-mail pour confirmer cette autorisation. Après confirmation, **Ghost** pourra envoyer des e-mails à ce destinataire.
7. Après avoir créé un compte propriétaire dans **Ghost** en visitant `http://localhost:3001/ghost`, et ajouté l'email du contributeur à la liste des destinataires autorisés dans **Mailgun**, vous pouvez maintenant envoyer un lien d'invitation au contributeur via **Mailgun** depuis `http://localhost:3001/ghost/#/settings/staff`.

8. Vous pouvez maintenant utiliser le lien d'invitation reçu dans l'e-mail pour créer un compte de contributeur. Une fois cela effectué, vous aurez une configuration identique à celle que nous utilisons avec la restauration de la base de données.

0.4.3 Tester l'attaque

Une fois l'installation réalisée, vous pouvez utiliser la commande suivante pour démarrer l'application :

```
docker-compose up
```

Pour tester l'attaque, suivez les étapes ci-dessous :

1. Générez la photo de profil SVG malveillante en utilisant le script `generate-malicious-svg.py`. Exécutez la commande suivante en remplaçant `[USERNAME]` et `[PASSWORD]` par les identifiants du compte de contributeur que vous avez créés :

```
python generate-malicious-svg.py -u [USERNAME] -p [PASSWORD]
-t http://localhost
```

Pour l'installation automatique, la commande est :

```
python generate-malicious-svg.py -u
malicious.contributor@company.com -p ZtXfMq2FaRHGm@!
-t http://localhost
```

Cette commande génère un fichier SVG malveillant nommé `tenant-takeover.svg` dans le répertoire courant.

2. Connectez-vous à l'interface d'administration de **Ghost** en utilisant les identifiants du compte de contributeur que vous avez créés, en visitant `http://localhost:3001/ghost`.
3. Accédez à la page de modification de votre profil en cliquant sur votre photo de profil en bas à gauche de l'écran, puis cliquez sur le bouton **Your profile**.
4. Cliquez sur l'icône de profil pour changer votre photo de profil. Sélectionnez le fichier SVG malveillant que vous avez généré à l'étape 1.
5. Enregistrez les modifications en cliquant sur **Save & close**. Vous pouvez rafraîchir la page pour vérifier que la photo de profil a été changée.
6. Connectez-vous avec le compte administrateur.

Sous `http://localhost:3001/ghost/#/settings/staff?tab=contributors`, vous pouvez voir que la photo de profil du contributeur a été changée. Si vous faites un clic droit sur la photo de profil et ouvrez l'image dans un nouvel onglet, l'attaque sera déclenchée. Vous pouvez maintenant revenir à `http://localhost:3001/ghost/#/settings/staff` et, après avoir rafraîchi la page, vous verrez que le contributeur est désormais propriétaire du blog et que vous êtes devenu administrateur.

0.5 Conclusion

La CVE-2024-23724 illustre les risques posés par une gestion inadéquate des fichiers utilisateur dans les applications web. En exploitant cette faille, un attaquant peut gravement compromettre un système, soulignant l'importance de valider et de filtrer les entrées utilisateur. Les administrateurs doivent agir rapidement pour appliquer les correctifs et renforcer la sécurité de leurs systèmes.

Références

- CVE-2024-23724 - Site officiel CVE Mitre.
- CVE-2024-23724 - Base de données NVD (National Vulnerability Database).
- Article associé à la CVE-2024-23724.
- CVE-2024-23724 - Vulners Database.